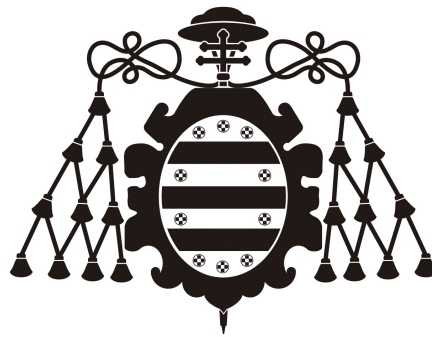


Aprendiendo PyTorch

Trabajo Fin de Máster
Máster en Ingeniería Informática



Universidad de Oviedo

Autor:

Rubén Martínez Ginzo, UO282651@uniovi.es

Octubre 2025

Índice

1. Introducción	4
2. Sistema de desarrollo	5
2.1. Sistema operativo	5
3. <i>Python</i>	6
3.1. Entornos virtuales	6
3.2. <i>pyenv</i>	6
3.2.1. Instalación de <i>ipykernel</i>	6
4. CUDA y drivers NVIDIA	7
5. <i>PyTorch</i>	8
5.1. Instalación de <i>PyTorch</i>	8

Índice de figuras

Índice de tablas

1. Introducción

En el marco de este Trabajo Fin de Máster (TFM), surge la necesidad de desarrollar soluciones computacionales de alto rendimiento aplicadas al procesamiento digital de imágenes. Con el crecimiento exponencial de los datos y la complejidad de los algoritmos modernos, es de gran importancia aprovechar las capacidades de las unidades de procesamiento gráfico (GPU) para acelerar las tareas computacionales intensivas. Este documento tiene como objetivo introducir los fundamentos necesarios para comprender el uso de *PyTorch* una de las bibliotecas de computación científica más utilizadas en la actualidad.

El enfoque principal de este documento se centrará en los tensores de *PyTorch*, los cuales constituyen la estructura de datos fundamental para representar información en esta biblioteca. Entender su creación, manipulación y operaciones básicas es imprescindible para avanzar hacia algoritmos más complejos.

2. Sistema de desarrollo

Para trabajar de manera eficiente con *PyTorch* y aprovechar la aceleración mediante GPU, es necesario contar con un sistema de desarrollo adecuado. Este abarca tanto el sistema operativo como las herramientas necesarias para gestionar dependencias y bibliotecas externas.

2.1. Sistema operativo

Aunque *PyTorch* ofrece compatibilidad multiplataforma, el ecosistema de desarrollo más utilizado para aplicaciones de computación acelerada por GPU es Linux. Las distribuciones de Linux proporcionan un entorno muy flexible y personalizable, ampliamente compatible con las bibliotecas y controladores necesarios para trabajar con *PyTorch*. A lo largo de este documento se asumirá el uso de un sistema operativo basado en Linux —concretamente Fedora Linux 42—, aunque las instrucciones pueden adaptarse fácilmente a otros sistemas operativos como Windows o macOS.

Entre las ventajas de utilizar Linux para este proyecto destacan:

- Facilidad de instalación de *drivers* y herramientas CUDA
- Compatibilidad con bibliotecas científicas de alto rendimiento
- Amplia documentación en entornos de computación científica
- Comunidad activa y soporte para desarrolladores

3. *Python*

3.1. Entornos virtuales

3.2. *pyenv*

3.2.1. Instalación de **ipykernel**

```
1 pip install ipykernel
```

Fragmento 1: Instalación de ipykernel en un entorno virtual de pyenv

4. CUDA y drivers NVIDIA

5. *PyTorch*

5.1. Instalación de *PyTorch*

```
1 pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu130
```

Fragmento 2: Comando de instalación de *PyTorch* con soporte CUDA 13.0

```
1 import torch
2 x = torch.rand(5, 3)
3 print(x)
4 print(torch.cuda.is_available())
```

Fragmento 3: Código de prueba de instalación de *PyTorch*

```
1 tensor([[0.6557, 0.0198, 0.0169],
2         [0.0468, 0.2888, 0.0026],
3         [0.4243, 0.1107, 0.6735],
4         [0.1006, 0.6152, 0.9174],
5         [0.1755, 0.9582, 0.8951]])
6 True
```

Fragmento 4: Salida esperada del código de prueba de instalación de *PyTorch*

```
1 pip install h5py
```

Fragmento 5: Comando de instalación de *h5py*